

Proyecto:

Efectos de la desigualdad socioeconómica en la
percepción de la democracia en Latinoamérica: periodo
1995-2023

Manual de Instalación

Versión 001

Versión	Causa	Responsable	Fecha
001	Versión Inicial	Patricio Mendoza	23/11/2025

Contents

1	Arquitectura general de la plataforma	3
2	Requisitos previos del servidor dedicado	5
2.1	Instalación de Docker y Docker Compose	6
3	Obtención del código fuente	7
4	Estructura de directorios del proyecto	7
5	Puesta en marcha de los servicios	9
5.1	Instrucciones de ejecución.	9
5.2	Configuración de dashboards y despliegue de <code>gateway</code> y <code>react_app</code>	11
6	Mantenimiento y operación	15
6.1	Backups	16
6.1.1	Backup de ClickHouse (volumen <code>clickhouse_volume</code>)	16
6.1.2	Backup de Superset	17
6.2	Logs	18
6.3	Resolución de problemas	18

1 Arquitectura general de la plataforma

La plataforma *Proyecto Democracia YACHAY* se basa en una arquitectura orquestada con Docker Compose, compuesta por varios servicios:

- **automatic_download** (descarga y limpieza inicial) Construido desde `./automatization/download_scripts`. Al iniciarse:
 - Monta volúmenes de datos:
 - * ENEMDU: `./data/raw/ENEMDU`,
`./data/enemdu_persona/{unprocessed,processed}`,
`./data/enemdu_vivienda/{unprocessed,processed}`.
 - * V-Dem: `./data/raw/VDEM`, `./data/VDEM`.
 - * Latinobarómetro: `./data/raw/latinobarometro`,
`./data/latinobarometro/normalized_csv`,
`./data/latinobarometro/processed`,
`./data/latinobarometro/unprocessed_dta`,
`./data/latinobarometro/unprocessed_csv`,
`diccionario latinobarometro_glossary_candidates_BM.csv`.
 - Utiliza variables de entorno de `.env` (`ENEMDU_ROOT`, `LATINOBAROMETRO_ROOT`, `VDEM_ROOT`, `PERSONA_UNPROC`, `PERSONA_PROCESSED`, etc.).
 - Ejecuta el script `runner.sh`, que encadena la descarga de ENEMDU, Latinobarómetro y V-Dem, así como las rutinas de limpieza inicial (normalización de nombres de columnas, diccionarios, etc.).
- **clickhouse_server** (base de datos analítica) Contenedor basado en `clickhouse/clickhouse-server`. Sus responsabilidades principales son:
 - Esperar a que `automatic_download` termine correctamente (`depends_on: service_completed_successfully`).
 - Montar el volumen persistente `clickhouse_volume` en `/var/lib/clickhouse` y los scripts de inicialización en `/docker-entrypoint-initdb.d`.
 - Configurar usuario, contraseña y base de datos a partir de `CH_USER`, `CH_PASSWORD`, `CH_DATABASE`.
 - Exponer los puertos:
 - * 8123: interfaz HTTP.
 - * 9000: protocolo nativo.
 - Ejecutar un `healthcheck` contra `/ping`.
- **clickhouse_client** (cliente interactivo) Contenedor `clickhouse/clickhouse-client` que:
 - Espera a que `clickhouse_server` esté `healthy`.
 - Ejecuta un `entrypoint` que comprueba repetidamente la conexión (`SELECT 1`) y, cuando está disponible, abre el cliente de línea de comandos de ClickHouse.
 - Se usa para ejecutar consultas de diagnóstico y mantenimiento manual.

- **automatic_ingest** (ingesta y cálculo de indicadores) Construido desde `./automatization/ingest`. Su función es mover los datos ya limpios a ClickHouse y derivar los indicadores:
 - Depende de:
 - * `automatic_download` (completado con éxito).
 - * `clickhouse_server` (`service_healthy`).
 - Monta volúmenes para:
 - * Diccionarios: `./data/diccionario`.
 - * ENEMDU persona/vivienda: directorios `unprocessed` y `processed`.
 - * V-Dem: `./data/VDEM`.
 - * Latinobarómetro: `normalized_csv` y `processed`.
 - * Logs y errores: `./automatization/ingest/logs` y `./automatization/ingest/errors`.
 - Usa variables como `CH_HOST`, `CH_PORT`, `CODIGOS_FILE`, `POVERTY_FILE`, `GEOJSON_FILE`, etc.
 - Ejecuta su `runner.sh` para:
 - * Ingerir diccionarios, ENEMDU, Latinobarómetro y V-Dem.
 - * Registrar el detalle de ejecución en `LOG_DIR` y errores en `ERR_DIR`.
- **superset** (BI / visualización) Contenedor de Apache Superset, construido desde `./automatization/init-scripts/superset`:
 - Depende de `clickhouse_server` (`service_healthy`).
 - Usa variables de `.env`: `SUPERSET_LOAD_EXAMPLES`, `SUPERSET_SECRET_KEY`, `MAPBOX_API_KEY`, `DATABASE_DIALECT`, `CH_HOST`, `DATABASE_PORT`, `CH_DATABASE`, `CH_USER`, `CH_PASSWORD`, etc.
 - Expone el puerto 8088 para la interfaz web.
 - Monta:
 - * `./volumes/superset/home` como `SUPERSET_HOME`.
 - * Scripts de inicialización y configuración (`init_superset_db.py`, `superset_config.py`).
 - Ejecuta un `healthcheck` sobre `/health`.
- **gateway** (backend Node/Express para *guest tokens*) Contenedor definido en `./web_app/backend`:
 - Lee variables desde `.env` mediante `env_file`: `./env`, incluyendo `SUPERSET_URL`, `SUPERSET_API`, `SUPERSET_USER`, `SUPERSET_PASS` y `ALLOWED_DASHBOARDS`.
 - Depende de `superset` (`service_healthy`).
 - Expone el puerto 8080.
 - Expone la ruta `/api/superset/guest-token`, que:
 - * Inicia sesión en Superset (`/api/v1/security/login`).
 - * Solicita un *guest token* para un `dashboardId` específico.

- * Restringe los IDs de dashboard a una lista blanca (`ALLOWED_DASHBOARDS`) que se actualiza con los UUIDs de los tableros a embeber.
- **react_app** (aplicación web de usuario final) Contenedor con el *frontend* React en `./web_app/frontend`:
 - Lee variables desde `.env`: `SUPERSET_API_FRONT`, `ALLOWED_DASHBOARDS`, `GATEWAY`, `MAPBOX_API_KEY`, etc.
 - Expone el puerto 3000, que sirve la interfaz web con menús, páginas de proyecto, metodología e indicadores.
 - Utiliza el componente `SupersetDashboard` para embeber dashboards, llamando al `gateway` para obtener *guest tokens* y conectando los IDs de Superset con las rutas/selecciones del usuario.

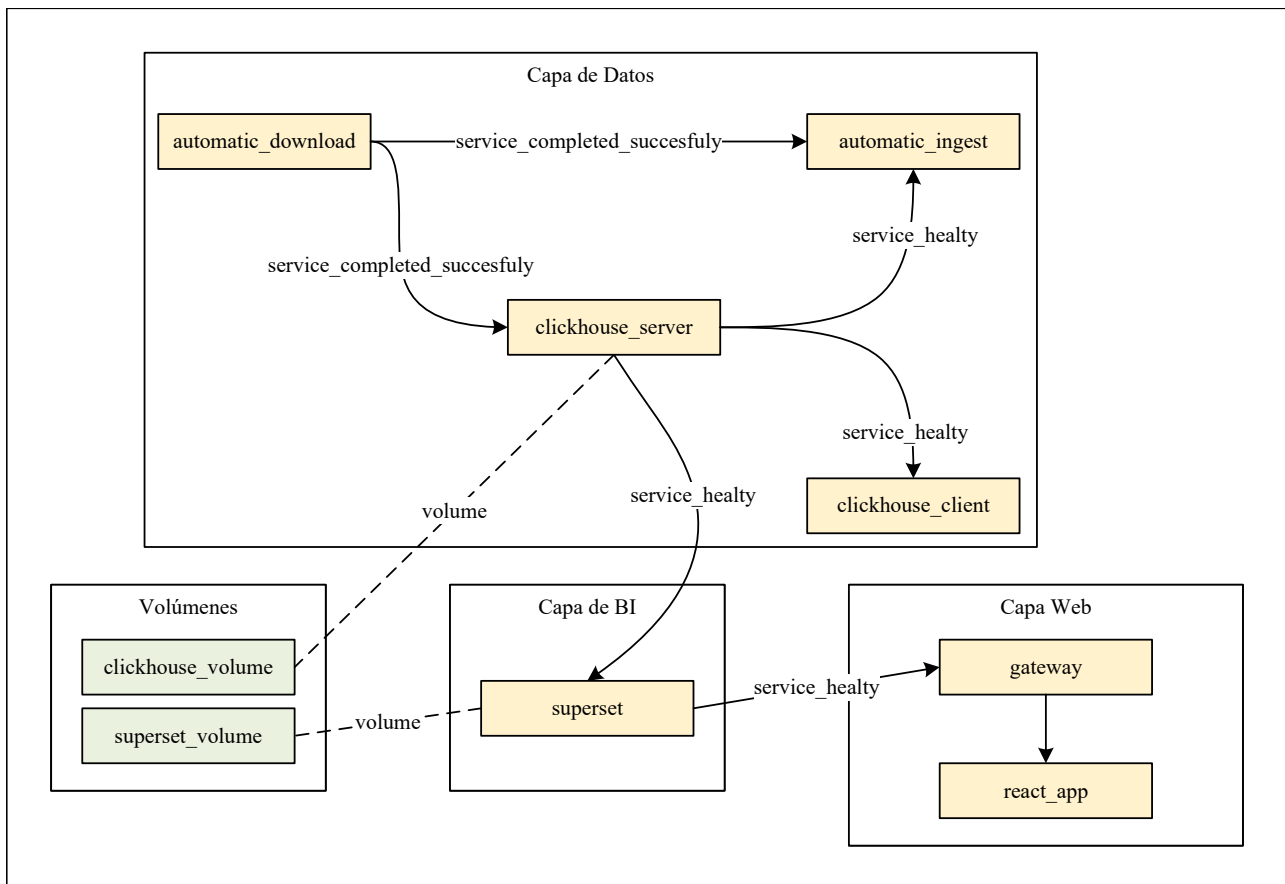


Figure 1: Esquema de los contenedores definidos en `docker-compose.yml` y sus dependencias.

2 Requisitos previos del servidor dedicado

- Sistema operativo tipo Linux (Ubuntu Server 20.04+ recomendado).
- Docker ≥ 20.10 instalado.
- Docker Compose ≥ 1.29 (o plugin equivalente en Docker moderno).
- Acceso a internet desde el servidor para descargar imágenes de Docker y datos de ENEMDU/Latinobarómetro/V-Dem.

- Usuario con permisos para ejecutar comandos `docker` y `docker-compose`.

2.1 Instalación de Docker y Docker Compose

En un servidor basado en Ubuntu Server 20.04+ se recomienda instalar Docker Engine y el plugin de Docker Compose desde el repositorio oficial de Docker para contar con versiones recientes.

1. Actualizar índices de paquetes del sistema:

```
sudo apt update && sudo apt upgrade -y
```

2. Instala los paquetes necesarios para agregar un repositorio HTTPS:

```
sudo apt install apt-transport-https ca-certificates curl \
    software-properties-common
```

3. Añadir la clave GPG oficial de Docker:

```
sudo mkdir -p /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
    | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

4. Añadir el repositorio de Docker para la versión de Ubuntu instalada:

```
echo \
    "deb [arch=$(dpkg --print-architecture) \
    signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu \
    $(lsb_release -cs) stable" \
    | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

5. Actualizar de nuevo índices e instalar Docker Engine:

```
sudo apt update

sudo apt install docker-ce docker-ce-cli \
    containerd.io docker-compose
```

6. Comprobar que Docker se instaló correctamente:

```
docker --version
docker-compose --version
```

7. (Opcional pero recomendado) Añadir el usuario administrador al grupo `docker` para no requerir `sudo` en cada comando:

```
sudo usermod -aG docker $USER
# Cerrar sesión y volver a entrar para que tome efecto
```

3 Obtención del código fuente

1. Clone el repositorio desde GitHub:

```
git clone https://github.com/datascienceyt/proyecto_pii25-08
cd proyecto_pii25-08
```

2. Verifique la estructura básica del proyecto:

```
tree -L 2
```

Debería observar carpetas como `data/`, `automatization/`, `web_app/`, `docs/`, el archivo `docker-compose.yml`, etc.

3. Otorgar permisos de ejecución a los `runner.sh` que se encargan de ejecutar los códigos de descarga e ingesta.

```
# En el directorio proyecto_pii25-08/ ejecutar:
chmod +x automatization/download_scripts/runner.sh
chmod +x automatization/ingest/runner.sh
```

4 Estructura de directorios del proyecto

El proyecto se organiza en una estructura de carpetas que separa claramente la lógica de automatización, los datos y los archivos de orquestación de contenedores.

<code>.env</code>	# Variables de entorno usadas por docker-compose
<code>docker-compose.yml</code>	# Orquestación de contenedores
<code>README.md</code>	# Descripción general del proyecto
<code>automatization/</code>	# Código de automatización (ETL)
<code>download_scripts/</code>	# Descarga y limpieza de datos en Python
<code>enemdu_descarga.py</code>	
<code>latinobarometro_descarga.py</code>	
<code>vdem_descarga.py</code>	
<code>limpieza_persona.py</code>	
<code>limpieza_vivienda.py</code>	
<code>limpieza_latinobarometro.py</code>	
<code>limpieza_vdem.py</code>	
<code>runner.sh</code>	

```

ingest/                                # Ingesta a ClickHouse
    ingest_persona.py
    ingest_vivienda.py
    ingest_latinobarometro.py
    ingest_vdem.py
    ingest_codigos.py
    ingest_geojson.py
    errors/                            # Filas con errores de ingesta
    logs/                             # Registros de ejecución
init-scripts/
    clickhouse/                       # Scripts SQL para inicializar ClickHouse
        create_table.sql
    superset/                         # Imagen y configuración inicial de Superset
        Dockerfile
        entrypoint.sh
        init_superset_db.py
        superset_config.py
data/                                 # Datos utilizados por la plataforma
    diccionario/                     # Tablas auxiliares (códigos, salarios, etc.)
    enemdu_persona/                  # ENEMDU (personas) procesado / sin procesar
    enemdu_vivienda/                 # ENEMDU (vivienda/hogar) procesado / sin procesar
    latinobarometro/                 # Latinobarómetro (dta, csv, normalizados)
    raw/                             # Fuentes originales (ENEMDU cruda por año/mes)

```

La carpeta raíz contiene los archivos de configuración generales del proyecto:

- **.env**: archivo de variables de entorno usado por **docker-compose**. Define credenciales, puertos, URLs y parámetros de conexión para los servicios (ClickHouse, Superset, gateway, frontend, etc.).
- **docker-compose.yml**: orquesta los contenedores de base de datos, automatización, BI (Superset) y capa web (gateway y aplicación React).
- **README.md**: proporciona una descripción general del proyecto y las instrucciones básicas de uso.

El directorio **automatization/** agrupa todo el código de automatización (ETL):

- **download_scripts/**: contiene los scripts de descarga y limpieza de las bases ENEMDU, Latinobarómetro y V-Dem. Estos scripts se empaquetan en el contenedor de *descarga automática* y generan archivos CSV normalizados en el directorio **data/**.
- **ingest/**: incluye los scripts de ingesta hacia ClickHouse y cálculo de indicadores a partir de los CSV ya limpios. Los subdirectorios **errors/** y **logs/** almacenan, respectivamente, las filas que no pudieron ser insertadas y los registros de ejecución de los procesos.
- **init-scripts/clickhouse/**: scripts SQL iniciales para crear tablas y vistas en la base de datos.
- **init-scripts/superset/**: archivos necesarios para construir la imagen de Superset y cargar la configuración inicial (usuarios, roles, conexiones a la base de datos, etc.).

El directorio `data/` organiza las fuentes de información y los datos procesados:

- `data/diccionario/`: tablas auxiliares (códigos geográficos, líneas de pobreza, salario básico unificado, geometrías en formato GeoJSON).
- `data/enemdu_persona/` y `data/enemdu_vivienda/`: estructuras con subcarpetas de datos procesados y sin procesar por año y mes, que alimentan los indicadores socioeconómicos.
- `data/latinobarometro/`: contiene los archivos originales en formato `.dta`, las versiones CSV sin normalizar y las versiones normalizadas usadas para la ingesta.
- `data/raw/ENEMDU/`: almacena los ZIP originales descargados del INEC, organizados por año y por ronda (mes), junto con documentación metodológica asociada.

5 Puesta en marcha de los servicios

5.1 Instrucciones de ejecución.

1. **Configurar el archivo `.env` en la raíz del proyecto.** Los parámetros para las rutas de archivos están pre-configuradas, pero elementos como nombres y contraseñas deben adaptarse para coincidir con las credenciales adecuadas. En el archivo `.env` en la raíz al menos los siguientes parámetros deben ser ajustados:

```
# ClickHouse
CH_USER=admin
CH_PASSWORD=ContrasenaSegura
```

```
# Superset
DATABASE_USER=admin           # Debe coincidir con CH_USER
DATABASE_PASSWORD=ContrasenaSegura # Debe coincidir con CH_PASSWORD
```

```
SUPERSET_USER=pmendoza
SUPERSET_PASS=pmendoza        # Usar una contraseña segura
SUPERSET_ADMIN_FIRSTNAME=Patricio
SUPERSET_ADMIN_LASTNAME=Mendoza
SUPERSET_ADMIN_EMAIL=pmendoza@yachaytech.edu.ec
```

```
# Web App (Nodej + React)
ALLOWED_DASHBOARDS=XXX        # Los UUIDs de los dashboards se obtendrán
                                más adelante
```

Estas variables son referenciadas por los servicios `clickhouse_server`, `automatic_ingest` y `superset` en el `docker-compose.yml`.

2. **Primer arranque: capa de datos y BI (sin gateway ni frontend)** Desde la raíz del proyecto, construir e iniciar únicamente los servicios de datos y visualización (usar `sudo` si es necesario):

```
docker-compose --env-file .env up --build \
    automatic_download \
```

```
clickhouse_server \
clickhouse_client \
automatic_ingest \
superset
```

3. **Verificar logs de automatización** Comprobar que los contenedores de descarga y de ingesta terminan sin errores:

- Descarga/limpieza:

```
docker logs -f automatic_download
```

- Ingesta:

```
docker logs -f automatic_ingest
```

Ambos contenedores deben finalizar con código de salida 0 una vez concluida la ejecución del runner `.sh`.

4. **Validar ClickHouse y Superset**

- (a) Verificar el estado general:

```
docker-compose ps
```

- (b) Comprobar la salud de ClickHouse:

```
curl http://localhost:8123/ping
```

- (c) Acceder a Superset en el navegador:

```
http://<IP-del-servidor>:8088
```

Iniciar sesión con el usuario administrador configurado en `.env` en los campos `SUPERSET_USER` y `SUPERSET_PASS`.

```
webapp@webapp:~/proyecto_pi125-08$ sudo docker-compose --env-file .env up --build automatic_download clickhouse_server clickhouse_client automatic_ingest superset
[sudo] password for webapp:
[+] Running 11/15
  ⚙️ clickhouse_server 9 layers [#####] 366B/366B Pulling
  ✓ af6eca94c810 Pull complete 16.8s
  ✓ e8787e2ab135 Pull complete 3.7s
  ✓ 4f4fb700ef54 Pull complete 4.5s
  ✓ db0c9a45f6b6 Pull complete 4.7s
  ✓ bb8ffa32619b Pull complete 10.6s
  ✓ e3f5546c736d Pull complete 10.9s
  ✓ 83f677c9c4cb Pull complete 11.1s
  ⚙️ d97cda2f5d33 Extracting [=====] 366B/366B 11.3s
  ✓ 246e85051bb7 Download complete 11.5s
  ⚙️ clickhouse_client 4 layers [#####] 205.1MB/235MB Pulling
  ✓ 2f94e549220a Pull complete 6.3s
  ✓ a72d8599d7c2 Pull complete 16.8s
  ⚙️ e9232762ed9d Downloading [=====] 205.1MB/235MB 4.5s
  ✓ 29c8f4b1e77e Download complete 4.7s
  ✓ 29c8f4b1e77e Download complete 13.7s
  ✓ 29c8f4b1e77e Download complete 2.5s
```

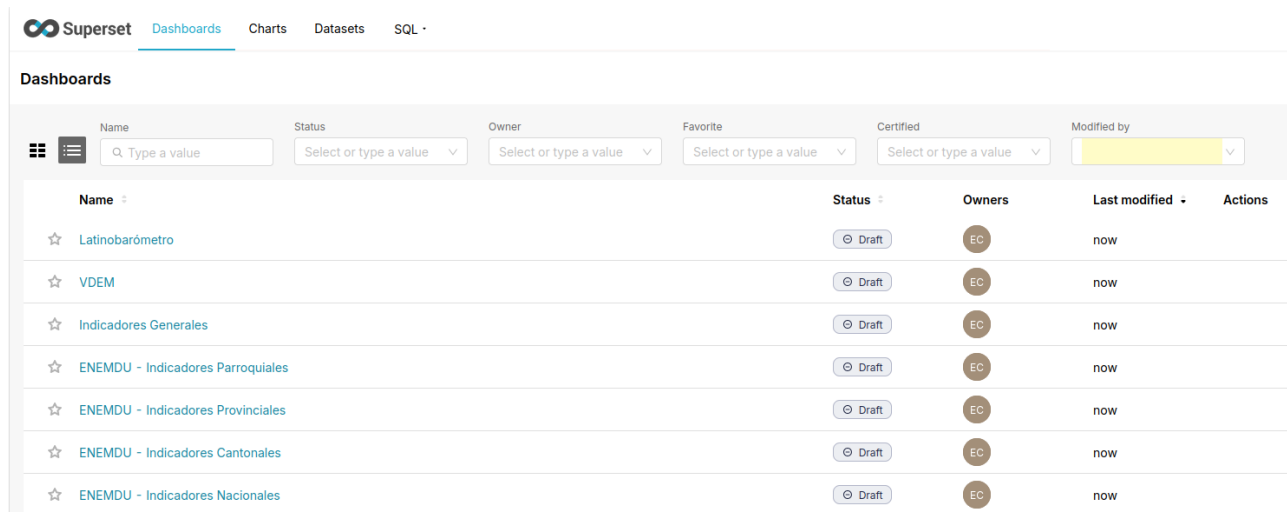
Figure 2: Inicialización de los contenedores básicos (datos y BI) tras ejecutar el primer `docker-compose --env-file .env up --build`.

5.2 Configuración de dashboards y despliegue de gateway y react_app

Una vez que la capa de datos y Superset están operativos, se deben seguir los pasos siguientes para conectar los dashboards de Superset con la aplicación web:

1. Crear y obtener los IDs de dashboards en Superset.

1. Ingresar a Superset en `http://<IP-del-servidor>:8088` con el usuario `admin`.
2. Crear o importar los dashboards que se van a exponer en la web:
 - Un dashboard de visión general (por ejemplo, “Visión general de indicadores”).
 - Tres dashboards comparativos por base: ENEMDU, Latinobarómetro y V-Dem.
3. Para cada dashboard:
 - (a) Abrir el dashboard.
 - (b) Hacer clic en el menú de opciones (tres puntos) y seleccionar *Embed dashboard*.
 - (c) En el campo *Allowed Domains* se debe colocar las IPs del backend (`http://<IP-del-servidor>:8080`) y del frontend (`http://<IP-del-servidor>:3000`) y hacer clic en el botón *Enable embedding*.
 - (d) Copiar el valor del *UUID* (es el identificador que se utilizará en `gateway` y en `react_app`).



The screenshot shows the Superset web interface. At the top, there's a navigation bar with 'Superset' and tabs for 'Dashboards', 'Charts', 'Datasets', and 'SQL'. Below this, the 'Dashboards' section is active. It features a search bar and several filter dropdowns for Name, Status, Owner, Favorite, Certified, and Modified by. A table below lists the dashboards with columns for Name, Status, Owners, Last modified, and Actions. The dashboards listed are Latinobarómetro, VDEM, Indicadores Generales, ENEMDU - Indicadores Parroquiales, ENEMDU - Indicadores Provinciales, ENEMDU - Indicadores Cantonales, and ENEMDU - Indicadores Nacionales. All are in 'Draft' status and owned by 'EC'.

Name	Status	Owners	Last modified	Actions
☆ Latinobarómetro	Draft	EC	now	
☆ VDEM	Draft	EC	now	
☆ Indicadores Generales	Draft	EC	now	
☆ ENEMDU - Indicadores Parroquiales	Draft	EC	now	
☆ ENEMDU - Indicadores Provinciales	Draft	EC	now	
☆ ENEMDU - Indicadores Cantonales	Draft	EC	now	
☆ ENEMDU - Indicadores Nacionales	Draft	EC	now	

Figure 3: Lista de dashboards creados en Superset para el proyecto (vista general y por base).

Embed [X]

This dashboard is ready to embed. In your application, pass the following id to the SDK:
[e1990a5f-26c5-4676-aa72-20f08167b698](#)

For further instructions, consult the [Superset Embedded SDK documentation](#).

Settings

Allowed Domains (comma separated) ⓘ

[Deactivate](#) [Save changes](#)

Figure 4: Propiedades avanzadas de un dashboard en Superset mostrando el UUID utilizado para el embed.

2. Registrar los IDs de dashboards en el backend gateway. En el backend Node/Express (`web_app/backend/server.js`) se define una lista blanca de dashboards permitidos:

```
const ALLOWED_RAW = process.env.ALLOWED_DASHBOARDS || "";
const ALLOWED_DASHBOARDS = new Set(
  ALLOWED_RAW.split(",").map((s) => s.trim()).filter(Boolean)
);
```

Antes de compilar la imagen de `gateway`, se debe actualizar el archivo `.env` con la lista de UUIDs separados por comas:

```
ALLOWED_DASHBOARDS=UID1,UID2,UID3
```

```
# Web App (Node.js + React)
# =====
PORT=8080
SUPERSET_URL=http://superset:8080
SUPERSET_API=http://superset:8080/api/v1/security
SUPERSET_API_FRONT=http://localhost:8080
ALLOWED_DASHBOARDS=e1990a5f-26c5-4676-aa72-20f08167b698,0971b53e-21a3-4a88-b589-e7a6313ffa4c,9fc78b88-d6a1-4008-a538-843937011421,eade6e74c-1583-4d86-97f3-6666b6f64a21
GATEWAY=http://localhost:8080
MAPBOX_API_KEY=pk.eyJ1IjoicG1lbnRvenFuIiwiaW50IjoiImNtaIdrbHJhNDAXeGQya29wYTl4a3ZtZWwlfQ.0rmERdS3oLT4ju_zaFu0iw
```

Figure 5: Edición de `.env` para actualizar la lista de dashboards permitidos (UUIDs) en el gateway.

3. Registrar los IDs de dashboards en la aplicación React (`react_app`). La página de indicadores en React (`web_app/frontend/app/src/pages/IndicatorsPage.jsx`) mapea cada “tarjeta” o modo de visualización a un `supersetId`. Dichos `supersetId` deben coincidir con los UUID de Superset:

```
// src/pages/IndicatorsPage.jsx

// 1) Dashboard general (único)
const GENERAL_DASHBOARD = {
  key: "general-overview",
  label: "Visión general de indicadores",
```

```

    supersetId: "UUID_GENERAL",
  };

// 2) Dashboards comparativos por base
const BASES_DASHBOARDS = [
  {
    key: "base-vdem",
    label: "VDEM",
    description: "Indicadores comparativos contruidos a partir de la base VDEM.",
    supersetId: "UUID_VDEM",
  },
  {
    key: "base-latinobarometro",
    label: "Latinobarómetro",
    description:
      "Indicadores comparativos contruidos a partir de la base Latinobarómetro.",
    supersetId: "UUID_LATINO",
  },
  {
    key: "base-enemdu",
    label: "ENEMDU",
    description: "Indicadores comparativos contruidos a partir de la base ENEMDU.",
    supersetId: "UUID_ENEMDU",
  },
];

```

1. Abrir IndicatorsPage.jsx.
2. Reemplazar UUID_GENERAL, UUID_VDEM, UUID_LATINO y UUID_ENEMDU por los IDs concretos de cada dashboard en Superset.
3. Guardar los cambios.

```
GNU nano 7.2 Indica
// ===== CONFIGURACIÓN DE DASHBOARDS =====

// 1) Dashboard general (único)
const GENERAL_DASHBOARD = {
  key: "general-overview",
  label: "Visión general de indicadores",
  supersetId: "e1990a5f-26c5-4676-aa72-20f08167b698",
};

// 2) Dashboards comparativos por base
const BASES_DASHBOARDS = [
  {
    key: "base-vdem",
    label: "VDEM",
    description: "Indicadores comparativos contruidos a partir de la base VDEM.",
    supersetId: "0971b53e-21a3-4a88-b589-e7a6313ffa4c",
  },
  {
    key: "base-latinobarometro",
    label: "Latinobarómetro",
    description: "Indicadores comparativos contruidos a partir de la base Latinobarómetro.",
    supersetId: "9fc78b88-d6a1-4008-a538-843937011421",
  },
  {
    key: "base-enemdu",
    label: "ENEMDU",
    description: "Indicadores comparativos contruidos a partir de la base ENEMDU.",
  }
];
```

Figure 6: Actualización de los `supersetId` en la página de indicadores de la aplicación React.

3.5. (Opcional) Configuración para entornos de VMs En caso de usar una VM se debe tener en cuenta que el localhost de la maquina virtual es diferente a la del host. En el archivo `src/components/SupersetDashboard.jsx` se hace referencia a localhost para la conexión con el backend y superset:

```
const SUPERSET_DOMAIN = "http://localhost:8088";
const BACKEND_GATEWAY = "http://localhost:8080";
```

Esto se debe cambiar para referenciar la IP de la máquina virtual:

```
const SUPERSET_DOMAIN = "http://<IP-de-la-VM>:8088";
const BACKEND_GATEWAY = "http://<IP-de-la-VM>:8080";
```

Nota: No confundir la IP del servidor con la IP de la VM.

4. Reconstruir y levantar gateway y react_app. Una vez actualizados `.env` e `IndicatorsPage.jsx`, se procede a construir e iniciar los contenedores correspondientes:

1. Construir e iniciar el backend gateway:

```
docker-compose --env-file .env up --build --no-deps gateway
```

2. Verificar que el gateway responde:

```
curl http://localhost:8080/health
```

3. Construir e iniciar la aplicación React:

```
docker-compose up --build --no-deps react_app
```

4. Comprobar el estado global:

```
docker-compose ps
```

5. Acceder a la aplicación en el navegador:

```
http://<IP-del-servidor>:3000
```

Deberían visualizarse las páginas de proyecto, metodología e indicadores, y al entrar a la sección de indicadores se embeberán los dashboards definidos en Superset.



Figure 7: Frontend React + Dashboards funcionando.

6 Mantenimiento y operación

El repositorio incluye un documento de operaciones y *runbooks* que describe procedimientos de backup, rotación de logs y troubleshooting. A continuación se resumen las operaciones básicas recomendadas para ClickHouse y Superset.

6.1 Backups

6.1.1 Backup de ClickHouse (volumen clickhouse_volume)

La base de datos analítica se almacena en el volumen Docker `clickhouse_volume`, montado dentro del contenedor en `/var/lib/clickhouse`. Una estrategia sencilla de respaldo es generar un `.tar.gz` del volumen desde el host.

1. Crear en el servidor una carpeta para almacenar los respaldos:

```
sudo mkdir -p /srv/backups/clickhouse
```

2. Generar un backup comprimido del volumen `clickhouse_volume` usando un contenedor temporal (por ejemplo, `alpine`):

```
docker run --rm \
  -v clickhouse_volume:/var/lib/clickhouse \
  -v /srv/backups/clickhouse:/backup \
  alpine sh -c "\
    cd /var/lib/clickhouse && \
    tar czf /backup/clickhouse_$(date +%F).tar.gz ."
```

3. Verificar que el archivo se creó correctamente:

```
ls -lh /srv/backups/clickhouse
```

4. (Opcional) Para restaurar el backup en un entorno limpio:

- (a) Detener los contenedores:

```
docker-compose down
```

- (b) Eliminar el volumen previo (si corresponde):

```
docker volume rm clickhouse_volume
```

- (c) Crear un volumen vacío y restaurar el contenido:

```
docker volume create clickhouse_volume
```

```
docker run --rm \
  -v clickhouse_volume:/var/lib/clickhouse \
  -v /srv/backups/clickhouse:/backup \
  alpine sh -c "\
    cd /var/lib/clickhouse && \
    tar xzf /backup/clickhouse_YYYY-MM-DD.tar.gz"
```

- (d) Volver a levantar los servicios:


```
docker-compose up -d
```

Nota: para escenarios más avanzados (backups incrementales, cifrado, etc.) se puede utilizar la herramienta `clickhouse-backup`. En este manual se describe la estrategia de copia de volumen por su simplicidad y portabilidad.

6.1.2 Backup de Superset

Superset utiliza dos tipos de información que conviene respaldar:

- El **volumen de configuración** asociado a `SUPERSET_HOME`, normalmente mapeado en el host como `./volumes/superset/home`.
- La **configuración lógica** (dashboards, datasets, etc.), exportable vía la CLI de Superset.

Respaldo del volumen `SUPERSET_HOME` Desde la raíz del repositorio (donde está `docker-compose.yml`):

1. Crear una carpeta de backups en el servidor:

```
mkdir -p /srv/backups/superset
```

2. Generar un `tar.gz` del directorio local asociado a `SUPERSET_HOME`:

```
tar czf /srv/backups/superset/superset_home_$(date +%F).tar.gz \  
-C ./volumes/superset home
```

Exportar dashboards y datasets vía CLI La CLI de Superset permite exportar dashboards y datasets a archivos ZIP dentro del contenedor. El siguiente ejemplo asume que el servicio se llama `superset` en `docker-compose.yml`.

1. Crear dentro del contenedor una carpeta de backups (si no existe):

```
docker-compose exec superset mkdir -p /app/backup
```

2. Exportar todos los dashboards:

```
docker-compose exec superset \  
superset export-dashboards \  
-f /app/backup/dashboards_$(date +%F).zip
```

3. Exportar los datasets (fuentes de datos):

```
docker-compose exec superset \  
superset export-datasources \  
-f /app/backup/datasources_$(date +%F).zip
```

4. Copiar los archivos ZIP al host:

```
mkdir -p /srv/backups/superset

docker cp $(docker-compose ps -q superset):/app/backup \
/srv/backups/superset/
```

Recomendación: incluir en la política de backup, además de los volúmenes anteriores, los archivos `.env`, `docker-compose.yml` y los scripts personalizados de `automatization/`, de modo que un servidor nuevo pueda restaurarse completamente a partir de los respaldos.

6.2 Logs

- Los logs de ingesta se almacenan en `automatization/ingest/logs` y los errores en `automatization/ingest/errors`.

6.3 Resolución de problemas

Procedimientos básicos (usar `sudo` si es necesario):

- Ver estado de contenedores:

```
docker-compose ps
```

- Forzar descarga y Limpieza de ENEMDU (Ecuador)/Latinobarómetro/V-Dem:

```
# ENEMDU
docker-compose exec automatic_download python enemdu_descarga.py \
    && python limpieza_persona.py \
    && python limpieza_vivienda.py

# Latinobarometro
docker-compose exec automatic_download python latinobarometro_descarga.py \
    && python limpieza_latinobarometro.py

# V-Dem
docker-compose exec automatic_download python vdem_descarga.py \
    && python limpieza_vdem.py
```

- Reprocesar únicamente descarga:

```
# ENEMDU
docker-compose exec automatic_download python enemdu_descarga.py

# Latinobarómetro
docker-compose exec automatic_download python latinobarometro_descarga.py
```

```
# V-Dem
docker-compose exec automatic_download python vdem_descarga.py
```

- Reprocesar únicamente limpieza:

```
# ENEMDU
docker-compose exec automatic_download python limpieza_persona.py
docker-compose exec automatic_download python limpieza_vivienda.py

# Latinobarómetro
docker-compose exec automatic_download python limpieza_latinobarometro.py

# V-Dem
docker-compose exec automatic_download python limpieza_vdem.py
```

- Ingresar al cliente de la base de datos y hacer cambios:

```
# Ingresar al contenedor de clickhouse_client:
docker-compose exec clickhouse_client sh

# Ingresar a la base de datos
# (Usar las credenciales reales definidas en .env):
clickhouse-client --host clickhouse_server \
    --port 9000 --user USER --password CONTRASEÑA
```

- Eliminar base de datos y contenedores:

```
docker-compose down --volumes
```

Nota: Al eliminar los contenedores y sus volúmenes se eliminará la base de datos. Los archivos CSV procesados de ENEMDU (Persona y Vivienda) y Latinobarómetro deben moverse a su respectiva carpeta "unprocessed" para que la ingesta se realice correctamente.

- Validar salud de ClickHouse:

```
curl http://localhost:8123/ping
```